

ActInf Livestream 055.1 ~ "Realising Synthetic Active Inference Agents" Part 1

Livestream | Part I & Part II | 1:43:43

Hello and welcome. It is November 1st, 2023, and we're here in ActInf Livestream 55.1 discussing realizing synthetic active inference agents. Really excited and honored to have Magnus here for this first discussion session. We're going to have an interactive presentation, and then looking towards the dot to have a bit more open-endedness and maybe some code execution, maybe some message passing live. So thank you for joining, Magnus. Very excited for your presentation and for this discussion. Thanks. Very, very happy to be here. Been looking forward with this for quite a while now. So should I just get into it? Yes. Right. Amazing. So hi, everybody. I'm Magnus. I'm a machine learning research engineer at Versus and in a couple of months, also a PhD graduate from Bios Lab in Eindhoven. I'm here to talk about these two papers, realizing synthetic active inference agents, parts one and two, that we did together with me and the ever brilliant, Thijs van der Lahr and Bert de Vries.

All right. So one thing ahead of time, if you have questions in the chat or anything, I can't see them, but I encourage people to interrupt me as much as possible, because it's more fun for everybody if we can still have a bit of a dialogue going, if we have our own action perception loop. All right. And with that out of the way, I clicked on my slides and they changed. So I think that part is working.

Excellent. So this is like the battle plan for what I want to go through today.

I want to start by covering the problem state and what are we trying to do with these two papers? What is sort of the whole idea and sort of serve you up the point, as it were, right at the offset?

Then we're going to cover a bit of background material. And then we start to get into the meat of the actual papers, which are mainly two things. It is what we call CFFG notation, a constrained phonestyle factor graph notation, and deriving messages based on local generalized free energy functional so that we can do this efficiently and scalably at all these nice things that we want.

And then at the end, we're going to look at some experiments that are in the two papers to see whether this is all just a nice theoretical exercise or whether we actually have something that works. All right, let's get into it. So the main thing that I want to do is we want to figure out how do we put active inference on graphs?

It's sort of the big question that we're trying to get at with these two papers.

Because active inference is awesome. I think you can say that in this crowd.

And graphs are really, really nice and have some nice properties that we would like for active inference. So let's get a little bit deeper into what this actually means.

Right, so I sort of put up two things side by side here. One being the active inference book, and the other being like a foreign style factor graph of a state space model.

We're going to try and compare sort of what we want out of each formalism.

So active inference, great, does a lot of things.

But one thing that it doesn't, in the very, very classical 2015 original formulations, is what I'm mostly going to be referring to. If we go really by the books, usually this is tied to a POMDP model structure. So that means we have these discrete state space models, we define them with A, B, C, D matrices. And everything is sort of, as we all know, on a lot. And this is great, we can do a lot of stuff with it. But it is still a choice that we make a priori most of the time. The other thing is, if we go strictly by sort of the original planning algorithm, there's a scaling problem in the planning horizon. That follows from us having to do these large search trees. There's excellent work trying to address this, either branching time active inference, parts of the sophisticated inference program, and all these really, really interesting developments using Monte Carlo Tree Search. But sort of, this isn't really a solved problem yet. On the plus side, what we do get, that we really, really want, is that if we design an active inference agent, and we go by all the nice things that we know and love, we get agents that are curious. We get agents that have this epistemic drive to go and explore their world, which is a really, really handy thing to have. Because if our agent can gather its own information, it will learn efficiently, it will sort of figure out how to navigate and do things. And it's much easier to have the agent do that, rather than have us as engineers, designers of the agent do that. Right. So to compare that to building an agent using box standards, racial free engine minimization on a graph, we end up with an agent that doesn't have exploration. If I take this type of formalism and I minimize free energy and I can use that to construct a controller, I end up with a KL control objective. And this is something that doesn't give us this exploratory drive. We can get all sorts of other nice things, but we don't get this information seeking behavior that is like a hallmark of active inference that we really want. The upside what we get is a lot more flexibility. I can write factor graphs for free-form models and by using message parsing tools. We'd also do inference in these models really, really efficiently. That means we scale better. Again, we have more flexibility in our model. We can do a lot of things, but what we don't get is this epistemic drive. So ideally what we would like is we want to take the best of both worlds. We want to take all the good parts of active inference and all the good parts of sort of factor graph message parsing based free energy minimization and squash them together to get one unified framework for doing all the stuff that we want. So that we get to get all the thumbs up and another thumbs down. Everybody still with me? Excellent. Okay. So to set this up, because these papers can get a little bit heavy. I'm going to try to walk through all the different parts, but to sort of set up the whole frame of how this is going to work, I'm going to start by reviewing a little bit of background material. It's not going to be too heavy, I promise. And we're going to start with our favorite functional, the variational free energy. Again, I think in this crowd, most of you will have seen something that looks like this before. And this is a fine way of writing it. We have some observations, x , we have some latents, z . I'm going to use x for observations and z for latents throughout. And we're interested in minimizing

this type of divergence. This is great. However, if we're in a position where we have, say, multiple latents combined in different ways and some parameters and maybe a couple of different states and things going on,

this particular way of writing things doesn't really sort of intuitively capture all of that.

This doesn't represent the underlying graph structure of our model.

So if we go up a little bit and look at this guy here, for instance, there's a lot of structure in this way of writing a generative model. See, some things are not connected to each other. Some things are. There's a lot of things going on here that we can't really capture if we just stick with this way of writing our free energy. So the first thing we're going to do is we're going to try and figure out how do we write free energy in graphs?

And to do that, we're going to have some graphs.

And I like this particular way of writing my probabilistic graphical models. There are others.

So this particular way is called a Forney-style factor graph. And basically, we have three building blocks. We have squares, which can represent factors in a generative model. So this is going to be, say, $p(x \text{ given } z)$. We have edges that we stick between squares. And they're going to represent variables. This is a bit different to what you would see if you do something like a Bayesian network, where a node would be a variable and you connect it to all the variables that depend on it. Here, the nodes are the factors and the edges are the variables. It's a little bit of mental gymnastics, if you're not used to thinking in this way.

And then, once we get into doing inference with message parsing, I'll use these little arrows to denote messages as they are flowing around on a graph. And all of these three building blocks is really most of what I'm going to use until we get into the meat of the two papers.

So let's look at an example of what this could look like.

So I have a pretty simple model here.

This has three variables, x , y , and z .

And they factorize like this.

So we have $p(y)$ gets its own factor, variable y , $z \text{ given } y$, this factor, variable z , and $x \text{ given } z$, this factor, variable x . So far, so good.

So again, this is just a simple graph. Now we're going to try and see if we can figure out how to do, distribute our free energy in this graph. How can we turn this general formulation that we had of just some latent relations into something that we can put all over this graph?

And this is the hairiest slide, I promise. We're going to do that by simply saying, well, I have my free energy, and I'm going to basically pick a term associated with each node. So I'm going to have a q for each node, and I'm going to have this, I'm using f for just a generic factor. You can also think of it as like the p argument in a normal free energy. And then we have this sum. So this sum here sums over all the nodes in our model. If you look here, we have one, two, three nodes. So that's going to be one term for each node.

And that's really all that's happening here.

So far, so good. This is not quite there yet. Because we're going to need another term, which is going to be this guy. And what this basically says is subtract the entropy of, add the entropy of each edge that is connected to more than two nodes back in. So why would we do this?

I promise you there's a reason, but this term, if you haven't gone through the ideas, seems a little ad hoc. Because now we have a free energy for each node, and we have this extra entropy term for each edge. So what's going on there? I want to sort of illustrate how that comes about, and we get to this form of the free energy, which is a Betti approximation to the free energy. We can look at this graph. You say, well, if we look at this node here, z given y , we're going to have a free energy term. And that free energy term includes two variables. This node is connected to two edges, so the free energy around this node will involve the variables that are connected to that node. That means we get terms for y and get terms for z . But if we look at the other node here, this x given z , that's also connected to two variables. It's connected to x on the one side and z on the other.

So if we do this summing up, do a free energy term for each node, this z is going to get counted twice. It appears once in the blue circle and once in the red circle. So how do we get around counting our variables twice? That's easy. We just subtract it again.

So that's why we get this extra entropy term coming in. That's why we get this guy. Because for each variable that is connected to two nodes, we're going to have this situation, because z is connected to two nodes, and therefore it gets counted twice. So we simply get rid of one of them.

Does this make sense? Beautiful. It's the same thing as saying that when we define q in our free energy functional, we have this $q \log q$ over p . We have a q that looks like this.

We say q of x , y , and z is going to be equal to all these terms, one for each node. There's one here for x and z , there's one here for y and z , that's one here for y , matching the three nodes, over these terms for the edges. And one for z , because it's connected to two variables, and one for y , because it's connected to two factors, and one for y , because it's connected to two factors.

So that's really all that's going on here. But if we do this, and take this Betti approximation to the free energy, now we can put the whole thing on a graph.

Because we have free energy terms for every node, and we have entropy terms for every edge.

So that means now I can take my free energy functional, and I can distribute it all over my graph. And I can work with everything locally. Because if I'm interested in updating z here, for instance, check down here, z appears here, q of x and z , it appears here, q of y and z , and then has its own marginal. So basically, if the graph were longer, I can throw away everything else. Imagine we have an I variable here, for instance, connected to y , that doesn't influence z in the slightest. They're completely separate on the graph, they've got a y in between, sort of Markov blanketing everything off. So I can take what would be large, unwieldy problems involving lots of variables, and just turn them into lots of small little local free energy minimization problems.

It's really nice, because we can solve them locally. And we don't have to deal with the whole thing,

we can just worry about all the little problems, we can string them together,
and get back reasonably good solutions to the thing we started with.
So that is sort of the main procedure for how we would put free energy on graphs.
Take the free energy, split it into terms for each node and each edge,
and then subtract whatever is double counted.
It gives the better free energy, really, really nice.
The other thing this allows us to do, which is something that
we go into more in depth in part two of the papers, is that we can do some variational calculus tricks
to derive algorithms that minimize this constraint B  the free energy.
And that allows us to say, set up the functional, formal Lagrangian, and get, excuse me,
belief propagation, or variational message parsing, or flash propagation, all these different algorithms
that we know and love, just as like from a general method.
And the spoiler is that when we go to part two, and we start to derive messages based on local
generalized free energy, we're just going to use the same trick.
And because as an engineer, I'm kind of a lazy person.
I don't want to remember 80 different algorithms.
I want to have one trick.
I want to do free energy minimization.
And I want to do it by message parsing.
And I want to have this method sort of on lockdown.
Because then my task is just to set up the right generative model and make sure that the
inference problems are right.
And then I can use that to do my smoothing or control my robots, or do my regression or
classification or whatever kind of thing that I'm interested in.
Again, I only have to learn one tool, which is really nice.
All right.
So, you are the only person on this call, Daniel.
So I'm going to be asking you, what's missing from this picture here of this graph?
If you think about your free energy functional, what's missing?
Art, hands, full, the list goes on and on, doesn't it?
There's a lot of, there's not a lot of humanity in these graphs.
This is like pure, pure engineering.
Like the thing that this graph represents really well is the generative model.
Now you can write your P really, really well.
You can't really write your Q .
Daniel Zeissner 1 No Q here.
Daniel Zeissner 1 Exactly.
Like this guy here, which is the other sort of half of the picture.

Think about your free energy functional Q over P .

You have a Q and you have a P , but we can only write one of them down.

The other half of the picture, the thing that actually specifies our inference problem, isn't really on this graph.

Daniel Zeissner 1

So, if I want to write this on my graph, I want to make sure that my graph is accurate representation of my inference problem.

I don't have to go and work out some equations or read through like a paragraph of text to figure out what's going on.

How do I do that?

Daniel Zeissner 1 And this is where we start in part one of the two papers.

So, one way you can think of defining your Q is that you can think of putting constraints on it.

It has to factorize in this way and it has to have this functional form and these things have to normalize.

Daniel Zeissner 1 There's a whole sort of set of mechanics that we can get into, but I haven't planned to, for how these can work.

Daniel Zeissner 1 Adding this to your factor graph effectively allows you to specify the inference problem.

Daniel Zeissner 1 Because you can write out the full functional with all the constraints and everything and minimizing that is going to give you the solution to the problem that you're interested in.

Daniel Zeissner 1 Okay, so have a generic graph

Daniel Zeissner 1 And we don't really have a way to put Q on it.

Daniel Zeissner 1 Write P , write my model, everything's great, but I can't really write Q .

I only have arrows, I have squares, and I have lines.

Those are the only things that I'm allowed to work with.

So to write Q , we need to introduce another piece of notation.

Daniel Zeissner 1 And the piece that we've chosen is this little guy, it's a little bead, a little circle.

And so I'll put variational marginal here in air quotes, just to see this represents some entry in Q .

So there's going to be one of these for every node, because we're using this best the approximation where everything gets distributed over the graph.

Daniel Zeissner 1 For every node and 1 For every edge.

Because we remember we had terms that involved marginals around nodes.

The model with x , y and z , we had one for x and z and for y and z .

And we're going to have little beads for every edge marginal, one for x , one for y and one for z . t because these are the things that we need to compute these double counting terms.

So if we take this generic graph from here.

And let's just add in beads that denote all the little parts of Q that we want.

We're going to be left with something that looks like this.

So again, what this is saying is I have a marginal here around this node.

node, this little guy, one on this edge, this little guy, and so on and so forth. And this just repeats for every node and edge in my graph. I want to be really explicit about it. And towards the end of part two, we do like a compression step because this gets unwieldy for large graphs, but this is sort of the idea.

A clarifying question here, Magnus. What values do the beads and or nodes and edges take on?

Like, we see the structural topological layout, and then is each bead going to be like taking on a number? Or is each bead taking on a variable that doesn't have to be simply a number?

So each bead here is going to represent a posterior belief. So the easiest way to think about it, there's going to be an entry in Q . So if we go back to this model here, for instance, we would have a bead that represents Q_z that sits here. So that would be this term, which could, for instance, be like a categorical or a Gaussian or a gamma or what have you, dependent on what falls out of your inference procedure. We have the same thing here, and we would have the same thing around here. Again, joint of X and Z , for instance, could be like a joint Gaussian. But these are going to be probability distributions most of the time.

Okay. So beads take on the semantics of the node or the edge that they're overlaid on, and the beads give us a little bit of a space between P and Q , quite literally, because it allows us to have this parallel Q -bead construction that inherits the structure and the relationships of the generative model drawn as a factor graph.

Exactly. And that's because we start by factorizing our Q over the graph. We split it into these nodes and edge-specific posterior marginal beliefs. That follows from the approximation, like the Betti thing we did initially. So that's where the structure sort of follows from. That's why we can put this across the graphs, because we started by saying, well, I'm going to distribute my free energy over my graph, and that's going to be my starting point. So then once we get into actually writing out the free energy, it would only consist of terms that are local to bits and pieces on the graph.

I don't know who asked that question, but that's a good one.

And again, I encourage people to interrupt me. If there are any more questions, let's just hit me now.

I'll be watching the live chat, but yeah, thank you.

All right. Excellent. Right. So the cool things about having these beads explicitly represented is that now we can start to do stuff to them. Like for instance, we can start to factorize things in different ways. We can start to talk about, well, should Q be forced to have a particular form? Like, should we force it to be a Gaussian, for instance? And that's sort of a lot of the meat of the part of one paper is just showing different ways you can do this if you want to write out, again, Q as well in your graph. Because if you have P and you have Q with the constraints, that tells you everything you need to know about the inference problem that you're trying to solve. That's the important part. Because now instead of having to write out stuff, text on equations, you can literally draw your inference problem as a picture. And to me, one of the most intuitive and sort of most practical set of moves

we do to Q is that we mean field factorize things. The mean field just means we say, well, I have my joint Q , where we say here, X , Y , Z , and V . And I'm just going to set it to be equal to the product of all the individual modules. So I turn this thing, joined over four variables, into this thing, with the product of four individual modules. That's literally all it is. And I want to represent that graphically. That means I take one beat for one queue, just the single one here. And I just split it into four beats, one for each queue. Because now it's really, I mean, to me at least, it's really easy to see that I have four different things going on. Otherwise, all the same rules apply. We stay inside the node because this is the node marginal, four connected variables. So these stay inside the node. And yeah, that's one thing we could do. But a full mean field in this way is also a bit aggressive sometimes. So maybe you don't want to factorize everything independently. Maybe you want to retain some joint structure in your marginals, which you can do. To do that, then you end up doing structured mean field, which I have two examples of here. So this is the same node that we had earlier. One giant queue joined over all the variables. But now I can, for instance, split it into two sets of two. I have a joint of X and Y and have a joint of Z and V .

So that'll be a joint of Y and a joint over Z and V . The trick here is that the edges that represent the variables, they protrude through the node border and sort of on that edge, we put the queue inside the node.

Does this make sense?

Yeah, I'll just restate it because I think it's one of the key contributions of the graphical notation.

So let's just say we were going to be drawing four variables, like rolling four die.

One way to analyze that statistically is treat all of them as being drawn like all at once from a four dimensional distribution that's unfactorized. And another extreme approach, aggressive approach, would be treat them as statistically independent or fully factorized. And so that was what was shown on the previous slide.

And now also there's nuance where you can say, actually, I think two of the die have a loose tether. So I want to infer the joint distribution of these two, but leave these two independent.

So now we can see and clarify, articulate all of the space between unfactorized and mean field factorized.

And then we can generate different graphs and asks like, well, which structural partitioning has what kind of computational or runtime complexity, what kind of accuracy? And so then you can engage in model selection across these different partitionings. However, seeing them all visually, as opposed to analytically, usually people are forced to just represent either the unfactorized or the fully factorized, because if you try to represent things analytically with all these partial factorizations on a large graph, it gets very unwieldy. Yeah, that's beautifully put. The idea is that we can treat some things as being joined and some things as being independent in queue. So the underlying generative model here is going to be the same. It's just how do we factorize our belief we want to do inference? That was beautifully put.

Right. So that's one thing we can do once we have an explicit representation for queue.

We can draw the beads that correspond to the structure in our aerational distribution.

But we can also put stuff inside the beads. I have like an empty circle. Same way I can write

something inside a node on my factor graph. I can write something inside my bead.

So what happens when we do that is we enforce different form constraints on queue.

Here I just picked one where I'm saying, well, the queue on this edge has to be a delta distribution.

So that basically means that when I do inference, I'm doing like the map estimate, the arc max of the variable on this edge.

And again, if you go through a bunch of derivations, you'll find out that that's what falls out from here.

So that means that now if I want to say,

you map estimate over my actions, if I have an agent, I can put that into my graph.

I don't have to write somewhere in my paragraph text, oh, and then we do this kind of thing and

we factor this way and then we take the map estimate. You'll know at a glance, just from looking at the graph that this particular variable on this particular edge has this functional form and therefore it has to be a map estimate.

You could do this with other things as well. You could say for it to be Gorshin or Wishart or something else if you wanted.

But this idea of using deltas is probably the most common that I know of.

Yeah, this is also very nice and very consolidating. The delta distribution is just a spike at one location.

And so it's basically a single data point or a value. So it's like a distribution that's kind of like a distribution as a handle on a data point. So the delta constraint is basically passing one data point rather than a full distribution through. It's actually a little, there's a little subtlety here.

Because you're completely right that if you have a, you know, a delta distributed variable center at some point, that's a way you can enter data. But that also means you need to fix the position of the delta distributed variable, which we do by calling it black. This is how you would represent data.

Otherwise, the position of the delta spike becomes a thing you can optimize. And that's why it becomes a map

estimate because it is still a point, but you can put the point in the best possible place, which ends up being the map.

So you're completely right. The form of the data point will also be a delta. But then you also have to fix where this delta is positioned. To distinguish between the two, we do data points, data constraints, as filled black nodes, and delta constraints, i.e. map estimates, as these unfilled nodes.

Does that make sense?

Yeah. Free delta versus a fixed delta.

Yeah, exactly. It's like saying you have to be only here. Or you have to be a point, but you can put the point wherever you want.

All right. Sorry.

Sorry about that.

Okay. So question. And once again, it's going to be on you, Daniel.

Daniel. Now we can put free energy on graphs.

And we can write out, you know, Q , we can write out the entire inference problem, including constraints.

So we're almost there.

What is the thing that we're still missing if we want to do active inference on graphs? Like, we want to go back to the original question, put the whole thing on a graph. What are we missing? The environment?

Yeah, that's one thing. Usually we would like blanket that off from our model. So that would live somewhere on the outside of our graph. But you're right, that would be somewhere that we would have to interface with an environment.

Daniel. But just in terms of the functionals that we want to optimize, there's still something missing. Expected free energy or anything prospective.

Daniel. Exactly. That's the whole deal. We've only been talking about variational free energy so far, and you can get really, really far with that. But one of the core things that we want out of active inference is we want to do planning. We want this information seeking, all this good stuff that we started with, the curiosity and the sort of interest in the world around you. So how do we put that on a graph? And that's the next thing that we're going to look at. How do we put this type of planning epistemic functional on a graph?

What do you know?

What do you know?

What do you know?

And to do that, we're going to look a little bit at how the EFE actually works.

So usually we denote an EFE by G , right?

That's like the common notation.

And we write that as a function of a policy.

So we write π , or in this case, I use u for my control variables.

So I have u from some time step T up until some planning horizon capital T .

So that means if I want to plan two steps in the future, capital T would be two and I would have two actions at either time step.

Does that make sense?

Awesome.

All right.

So the first thing we're going to say is, well, we compute that by taking a sum of the free energy at every time step.

That's the standard way to do it.

If we expand that a little bit,

we get a sum over risk term and an ambiguity term, just like we all know and love.

So I have to worry about that.

But the key part,

the one that usually doesn't get as much attention in this part here, usually we like to look at here.

I have the sum.

Because the sum tells us something important about the expected free energy.

It says that it's the sum of a bunch of independent terms. So this function actually factorizes.

Now, if you think about it, it means we have one term for each time step,

or if you write out the full graph, there would be one node on our graph.

And to compute the whole thing that we're interested in, we just sum them up.

So the actual objective factorizes over the graph.

And that's kind of neat, because it means that when we look at the way this is constructed, like from the get-go, there's already sort of a hint of how we can go about putting it on a graph.

And what we would need to do is figure out, well, if I want to do this around one node on my graph, how do I do it? And do it in a way that everything is consistent and we recover all these nice objectives and everything that we've already seen before. So how do we do that? How do we make a functional that's local to a node that inherits all the nice properties that we want out of an expected or, spoiler alert, a generalized free energy?

And we're just going to go through and construct it.

So we're going to start, because we're working under the umbrella of the free energy principle, with a variation of free energy. It's going to be our first term here.

And it's going to be for some node that's connected to two variables, x and z .

And then we also know that a lot of the nice epistemic drive that we see in active inference agents, it follows from some information gain term.

So we're simply going to bolt on an information gain term that's local to our node.

And if we do that, we can do something quite interesting then.

So are you up for going through the duration?

Yeah.

All right. So the first thing we're going to do is we're just going to pull everything into the same integral. So both of these guys integrating over q of x and z , q of x and z . That means we can put them together, just do it once.

Then we look inside these brackets and go, wait a minute, we've got two logs.

We can pull all this inside the log.

I think nothing fancy here.

But then we look at this guy and we go, wait, I have a q like either side of my line.

These two guys are going to cancel.

Divide something by itself.

That's really, doesn't really give you much.

And if we do that, then we're actually left with the generalized free energy.

Local to a node, similar to what Thomas Parr and Fristen introduced in this 2019 paper.

So that means if we take our free energy around a node, which we can do because we've already gone

through all the rationale for why that's a good idea, and we bolt on a negative information gain term, we actually end up with the same functional that was already proposed in this amazing paper like four years ago.

That's kind of nice.

Could you maybe express what is it about the negative information gain that's doing the work, the epistemic work?

Right.

So the way I like to think of it, first of all, it's negative because we're minimizing.

We really want to maximize information gain, but we're minimizing our free energy or EFE or whatever.

So we flip the slide.

The other thing is you can rewrite this into, say, a marginal entropy over your states, your Z , minus a conditional entropy over Z after you've seen X .

So what does this mean?

It means one of these terms is going to encode how much uncertainty is in your latent variable, your latent state Z before you do anything.

How uncertain are you right now about something?

And what you subtract from that is the uncertainty after you've seen some observation X .

And the bigger that difference, the larger your uncertainty before getting an observation, and the lower it is after, the higher the information gain.

So that's how I like to think of why information gain sort of gives us this exploration thing.

Because when we allow our agents to pick out observations,

we allow them to pick what we should condition on when we calculate the information gain.

And we tell them this is what you're supposed to optimize.

They're going to go look for the places in the world where they are most uncertain about what's happening and where they predict that if they observe something, they're going to be the most certain.

And how would you contrast this expression with the decomposition of expected free energy as pragmatic value plus epistemic value?

So that's the beauty of it, right? I don't actually have to.

I don't think so.

Because what they, what Paul and Fristen also showed in this paper is that if I do this

functional on the same POMDP model that we use for everything else, it actually evaluates the exact same thing as the expected free energy.

So for all intents and purposes, it is exactly the same.

So all the intuitions about EFE are going to map.

Maybe this is looking a little ahead, but do you foresee that generalized free energy will come to rule them all?

And that we won't talk so much about VFE and EFE? Or do you see those as still holding down important semantic space?

I think it's an open question.

I'm not confident enough in any of this, and especially not about the future, to know how things are going to turn out.

But I do know that if I take this maneuver, if I have a variational free energy, which I can use to recover something like KL control,

and I bolt on an information gain term that gives me this exploration that I like, I can end up with a generalized free energy.

And if you go through the proofs in part two, you also see that this makes it easier to derive messages compared to an expected free energy because we tried the expected free energy as well, and it didn't really work.

So for this particular application, the generalized free energy works better.

As for any sort of ontological status, I'm just going to be an engineer.

What about explicitly pragmatic aspects of future time steps?

So if we have the VFE embodying the real-time pragmatism, and then we have an explicit future-oriented epistemic component,

where would we find that kind of classical EFE flavor of pragmatic value of future states as well?

Can I come back to that in a few slides?

Oh, of course.

Because I actually have an example of what this construction gives you if you want to add in,

say, a goal prior. So you want to add C vector in the standard nomenclature,

or basically like a prior over future desired observations.

Yeah.

And that's the thing that's going to power the goal-directed behavior and the experiments as well.

So the spoiler version is that this ends up, if you work through it and you cover, like, take this node construction that we came up with, and you work out the corresponding energy term that you would use to calculate, say, a pathy free energy, you end up with the exact expression that you usually get for expected free energy.

So you can actually recover one-to-one the standard sort of 2015 algorithm

by writing this in your graph and doing these manipulations, and then doing it in a particular way. You will get the exact same thing back.

But yeah, hold on to that thought, because in a minute or two, we'll get back to it.

So, okay, yeah, this is another thing. So there are a couple of ways to go through this.

And to make it exactly like, I was saying, to make it exactly like the thing we find in, say, classical active inference papers, we need to do this kind of substitution here. So here,

I've basically gone to the same, get to the same results, but in a different way,

where I mean field factorize, and then we exchange this Q for this P. That's important.

So usually, if you read an active inference paper, you see this P of usually O and S and Q of S.

This joint here denoted with something like Q with a tilde above it.

And it just means we substitute in this particular part of our generative model.

So if you go through the standard tutorial type things, this would be substituting in your A matrix,

for instance.

But we call this move a P substitution, because we're literally substituting a P for a Q.

And we do this because, not because we need to, in order to derive all the messages that we need to be able to do this in a nice distributed way, and get to our end goal of putting active inference on graphs.

So I just want to introduce this move here, because it's the last bit of notation that we're going to need to cover basically everything that's in part one. And the way we write this is literally,

if Qs are circles, and Ps are squares, we are going to swap a circle for a square.

So we take the little bead here that represents a Q, and we swap it for a square. And this square means, in the expectation of your node local free energy, substitute P of this variable, conditioned on everything else that's connected.

So you can see if we have a mean field factorization here, Q_x , Q_c , Q_x , Q_c , and then we do a P substitution, we are left with the same generalized free energy function. Just a different way of getting there.

Does this make sense?

Awesome.

Okay.

Okay.

And that is like the main meat of what we introduced in part one.

So sort of to recap everything we've done so far, because we're moving at quite a high pace.

We've taken our free energy, and we put it on graphs. We did that by using a Bethy approximation, so we have a Bethy free energy.

Okay.

Then we realized that we can't write down everything that we need to, in order to specify everything in our inference problem. We can't write down Q, and we can't write down our constraints on Q.

So we came up with CFFG constrained for a new style factor graph notation, to write down the entire inference problem graphically.

Then we figured out that the EFE factorizes, which is great,

because that means we can also treat it as something that's local on our graph.

And if we take node local free energy, the kind of thing that would give us KL control if we did it everywhere, and we bolt on an information gain term, we recover something that looks an awful lot like generalized free energy that Friston and Parr derived in 2019, which numerically ends up,

I got to break the ends up evaluating to the same thing as the expected free energy.

So we have a version of expect for these pumply P models, I should say. So we have a version of the expected free energy that is also distributed over our graph.

Is there anything that is relevant to mention about discrete versus continuous time models?

Yes, so we haven't done any of this work on continuous time models, like these large generalized coordinate filters with LaPrasse approximations. We haven't done that here, because the base representation we're using is a graph. So when we want to do planning, we literally have to write out explicit time steps for the future. You could probably do something like this for the whole continuous

time active inference as well. But that's a whole other can of worms that we haven't dug into.

Yeah. So, so far, this is only something that's been worked out for the discrete time case. It doesn't have to be discrete

state spaces. You could work this out, because we get the general form of all the messages. You could work this out for continuous variables if you want. But yeah, continuous time is not something we've done yet.

All right. I mean, if there are any other questions, I think this is a good time to pause,

because we're going to get into the next sort of meaty section in a minute.

I'll ask one question from live chat. Upcycle Club wrote,

how can synthetic active inference agents be integrated with other intelligence paradigms,

such as deep learning or evolutionary algorithms?

So, the beauty of how this ends up working is that this becomes another message-passing algorithm.

Not even another message-passing algorithm becomes like literally one sort of local message out of the nodes

that we do these operations to. Everything else is going to function like box standard, VMP, or belief propagation, some product, sort of classical machine learning tools.

So, anyway, you can usually combine your probabilistic graphical model with a deep neural net. Like, you'd have a deep neural net powering your transition dynamics, for instance, if you wanted.

And that wouldn't change anything here, because we're still only doing stuff locally on a graph.

So, if we still actually have a graph here that shows something.

I'm just going to go back to one of the early graphs, because it's probably the easiest way to illustrate.

Yeah, this guy here. Let's say that we wanted to do all these operations around this node here, where we have like a latent Z , and we connect to some outside world that we call X , where we get all observations. All operations are going to happen inside the blue circle.

Everything that we do is actually only going to affect the variables that are connected to this node and around this node. So, everything else, everything that's over here, is going to work exactly the same way as with any other message-passing algorithm. So, you could hook this up to, say, a deep neural network, if you've trained it right, and use that to, say, be a node function somewhere.

I suppose you could also use it with evolutionary optimizers. It's not something I've seen done a lot, but it's a cool idea. And I don't see any reason in principle why it shouldn't work. Because what we're doing really is just changing the objective functional around one of our nodes a little bit.

Does that answer your question, Upcycle?

Cool. Continue.

All right. Let me get back to where we were.

Whoops. Here. Right. Okay. So, this is where we're going to look at how we add in something like a goal prior. Because we had this like a little while ago. How do you get your goals and your constraints and your objectives and your desires and your agents' desires and life, as it were, into the model? How do we do that?

And the way we came up with doing it is that we defined this composite node.

So, this is going to be, again, hold on to your butts. This looks nastier than it is.

Really, what we're saying is we have a box on our factor graph. Because again, all we're allowed to do our lines and boxels and arrows and beads. But we have a box on our factor graph. And inside this box, there's going to be two other boxes. We're going to have this guy here called P , which could, for instance, represent a likelihood model. And we're going to have this guy here called $\tilde{P}$, which is going to represent our goal.

And that means when we go through the motions of defining

Q around this composite node with all our modifications and everything, we end up with a model, like a P , that is not normalized. It's going to be P of X given Z , right?

Yeah, P of X given Z , it's going to be this node, times $\tilde{P}$ of X given W , in this case, if it has any parameters.

What it basically means is that we can add in a goal prior to our model, at the cost of it being not normalized.

It turns out that it's not really a problem that's not normalized.

But the basic idea is that if we want to add in goals, what we do is simply we wrap our goals and our likelihood into one box on our graph. And we just go through the motions.

You just sit down, you calculate, you work through all the little bits, pieces, and you push all your symbols around. And at the end, the result you're going to get is going to be something useful.

So this is the way to adding goals. You simply just bolt on an extra factor to your model that encodes your goals.

Yeah, I don't know if this is helpful, but check me on this. It's almost in like a racing game, when the high score has the ghost that is leading the pack. So we have like the goal prior is kind of like the ghost in the midst, and then there's all the actual P , but then there's the shadow and perfect alignment or perfect fulfillment is when the actual, there's no space between the actual and the preference. So in the textbook, analytical representation, that would be like P of O conditioned on C .

Yeah. So we have the same, it's just P of X conditioned on W .

But it works exactly the same. I think that's also what we call it in the experiments. If you go through it, it ends up just being the normal C vector. But you're right, you're completely right.

If you're doing a racing game and you have say an idealized perfect run as your ghost, that would be going in here. And you would sort of map out the whole trajectory of where the ghost is at all the time points you want to plan over. And then from here, from the top, from the Z 's, is going to be your attempt to recreate the perfect run.

Yeah. Which is actually an interesting idea. You could have some kind of expert trajectory, define your goals, and then use that to learn the parameters of your model, then do your planning from that. That would actually be a cool thing to try and do.

But yeah. I think so far, the intuitions are like on point.

All right. So I'm just going to keep going, I think.

Because one of the questions that came up during the .zero livestream

was pull up a lemma and be like, what does this mean? Pull up a lemma, be like, what does this mean? And so on and so forth. So I'm taking the liberty of just putting on this graph what all the lemmas mean. Because then we can talk about what they mean. And if you want to go through the nitty gritty derivations, they're there in the paper. Because, again, as I mentioned earlier, I'm a laser engineer, I really only have one trick. That is, I can write out my free energy and I can solve for stationary points and that gives me message passing algorithms. And that's all I do. So if you look at this graph here, this node, there's basically three things that we need to solve for. We need to solve for x . What's the stationary point here? We need to solve for z . And by symmetry, that also applies to θ . If this likelihood node has any parameters, for instance. So you could think of this as being like a latent state and an A matrix with like a Dirichlet prior. So we need to solve for w , which is the parameters of our expert run, like our c vector in the standard nomenclature. And if this node has any parameters, like whatever they may be, usually the standard form, it doesn't. But you could. So really, there's three things we need to solve for. This guy in the middle, stuff going out of this node, and stuff going out of this node. And it just so happens that there's three lemmas in the paper. There's one that solves for this guy. How do we update our predicted observation? Which ends up coinciding with how we would usually do it if we're computing EFE, which is nice. There's another lemma that says, what should the backwards message towards our state be? And this is going to be the important one. This is what allows us to do a lot of the really cool stuff. Because this basically says, if I'm optimizing GFE, EFE, and I want to reason backwards from the best possible outcome in terms of balancing all epistemics and everything, what should that be? That's basically what comes out here. And we have a third lemma that deals with the stuff coming out the other way. Let's say you want to update your goals. Let's say when you drive your run, the ghost sort of reacts to what you do. That's kind of the thing that would be coming out here. And then we have two theorems that show that the messages coming out here and here, if you iterate them, the stationary points can actually be found by fixed point duration schemes, i.e. you can solve this by message passing. So to recap, the lemmas are what should these guys look like? And that's where you find the messages. And the theorem states that if you do this, you can actually solve your problem and find these stationary points by fixed point iterations. And this is a little technical. And if you want to get into it, I encourage you to go and read the paper. But the point being, I'm still a laser engineer. I still only have one tool. I can only solve for stationary points of my free energy and my graph. And if I do that, I have three things to solve for. And I have to make sure that if iterated, I also get something sensible out. And that's what the three lemmas and the two theorems are about. Very nice. So factor graph, not just as telecommunications, message passing, infrastructure,

but as research agenda and conveyance heuristic.

Yeah. Once you're in the space, sort of this way of writing things like graphically, you can write your problem graphically and it's easy to see how two things are different. And you can also write something where you don't know the solution yet.

If you don't know the solution, that's a research problem.

Yeah. And the dash lines and then it's a great palette.

Thanks.

Okay. So this is the first thing that I hope is also going to end up being useful. Because again, what I'm doing really here is that we're presenting tools. We're building stuff for other people to hopefully use. And sort of the litmus test as to whether this is going to be actually practical for people who are interested in building active inference systems and reasoning about active inference systems is whether the people that are interested in this actually end up using it. Otherwise, it's a neat idea that just ends up living in a drawer.

So I have an open invitation to everyone. If you're interested in using this kind of notation, I'll be happy to help you with your graphs. Because I think it would be a benefit if we can all agree in a unified way of specifying our problem. And it should be one that is not just geared towards experts. I should be able to write out all the equations and everything. Now, that should also be something for us. But if you are interested in applying active inference, you should also have an easy sort of intuitive syntax for how to write down exactly what it is you're doing.

Because if you do that, then everyone knows. And then communication becomes easy. And writing out your algorithm becomes easy. And so making sure that everyone knows at all times what's going on becomes straightforward because you can literally read it all from the picture. So again, an open invitation to anyone who's interested in this, hit me up. I'll be happy to take a look at your stuff. That's awesome. I think one more piece I'll add. That's a very prospective, very pragmatic point to make. And also, I believe that there's a great opportunity for meta analysis of literature and re-derivation of these kinds of graphs for the figures that we've seen over the last 15 years. And then that will make a lot of continuity because we can then bring the factor graph motifs from models that are widely discussed and kind of bring those into the next generation. That's the hope. And it's funny that you say that out of all things, because if you look at what goes on the next slide, that's actually what we do.

Next. So this is a way of drawing a standard, you know, anno 2015 active inference EFE evaluation as a constraint for an style factor graph. So let's break this down and see what it does.

Because I mean, I've been talking for like an hour already and I've been talking about nothing.

Most of what I've been talking about is what all these boxes and little bits and pieces mean.

So if we start at the very end, and I hope I'm pointing in the right direction. I don't know how I'm mirrored in the screen, but otherwise it's that end. One of the things we would use to define is a prior over where our agent starts, which we denote by a D vector. That's the standard way of doing it.

So we want to add in a D vector. We put this little square because there's a variable that's clamped in P. There's like a parameter that we don't mess with.

You could update it and you could indicate that in a graph, but here we don't.

And we have this little guy. That's because

these two are going to be different. We don't have a joint of D and Z. We just have a Z.

We move on. Now we have this node, which I've labeled with T. And T is for transition. This indicates a discrete state transition, i.e. a multiplication by a B matrix. The B matrix is what we find here.

So what this is saying is we have fixed an action at time T

in order to move from Z at time T to Z at T plus one. It's just like a multiplication by transition matrix.

Then we have this thing. This is an equality node. Why do we have an equality in the middle of everything?

And that's because we're using edges to represent variables. And an edge is a line. I can only connect an edge to two things. So if I need my variable to connect to more than two things, I need to create copies of my variable. If I want to create copies, that's the same as saying I have two things,

three things, and they're all going to be the same. They're all going to be equal. So this equality

node says this edge here and this edge here and this edge here are all going to be the same. That's all

it does. And we can keep going here from Z at T plus one with a new action, new transition, Z at T plus

two and so on and so forth. That basically just applies these transition matrices. But the really

interesting part is happening down here around what would be a likelihood node. Again, we have this

multiplication by a matrix. In this case, it's going to be a likelihood, our A matrix. We have this

composite node construction where we have a goal prior, it's a categorical parameterized by Z.

And we have our observations here. And so this construction here is what, oh yeah, we have this

P substitution, we have these two guys factorizing separately. So we have all the constructions

necessary to do our local GFE, which in this case equals the VFE, sorry, the EFE. We have our goal prior,

we have all the little ingredients and bits and pieces. So how do we write, given that we have all

this, the exact same algorithm as people are using 2015? We do that by adding in arrows. Remember, I had

arrows as the last thing way back when we started. And arrows in this case describe the schedule, i.e.

Which messages do you pass? And you can sort of follow these along and see that you're kind of

you're rolling forwards through time.

So this is like creating a rollout and saying, well, what would happen if I did that at this time?

Well, go here. Then what would happen if I did that? Go here and so on and so forth. So you get

this sort of forward to roll as a function of a policy, i.e. the series of matrices you fix up here.

And this composite node, energy term of that node ends up being exactly equal to

one term of the EFE, GFE, this model that we already established factorizes over time.

So you roll this thing out, do your forward rollout, take all the energy terms of these composite nodes and

sum them up. And that gives you back the exact algorithm that's used to evaluate EFE in like the

very original first types of papers.

So just to make a clarifying point and then a question. So for comparison to the Bayes graph and

the 2022 textbook, people might want to check out figure 4.3 or figure 7.3.

So here we have already clamped B. So here we're just evaluating one policy because we see B as a black square.

Right. Because we need to go by the sort of standard version. We need to evaluate the EFE as a function of a policy. So we put in a policy, we do this rollout and we read off our EFE number. And then we take the soft max of all EFE's and it gives us a distribution over policies that we can select from.

And it's this rollout thing that makes the scaling hard because every time you add a new step, you just go through all your possible actions and the search tree branches and to get into all these things that people are trying to solve in a whole host of different interesting ways.

But you're right. We're treating policy as something that's fixed because we're evaluating EFE as a function of a policy.

Yeah. And then of course you iterate over your affordances, over your policies to calculate EFE for each. And then you get policy posterior and then you can do what you will with the posterior.

Yeah. So then it basically becomes like a model comparison problem. Given this policy, this rollout, and I get this energy terms, which one is the best?

But it's funny that you say that because one of the things that we also do now that we have backwards messages is that we don't have to fix the policy.

All right. Think about it. All the stuff we're doing happens down here in the little sort of dashed box.

But the policy lives up here. We've already established that if everything is local in the graph, I can do whatever I want down here. It's not really going to impact anything up here.

So if I now turn these black squares into random variables, I can still do inference.

And I can still do inference in a way that gives me back a policy.

However, now this policy is going to be one that optimizes GFE or EFE locally in these nodes. Otherwise, everything is going to work as if you were doing like a forwards, backwards, KL control type thing.

And what we need to do then is we need to take all our candidate matrices and we put them into this mixture model.

And that basically says, if I have my indicator variable, the indicator variable picks out one of the transitions.

And then you want to infer the posterior of your indicator variable such that that it becomes a good approximation for the probability of selecting each particular matrix at this point in time.

So that did not come out clear.

So that makes sense at all.

So that makes sense at all.

Yeah.

Maybe explain that in the context of why it's called TM.

It's TM for transition mixture.

So it's like you have, instead of saying, I do either, you know, action one, action two or action three, I sort of do a mixture of all of them.

And then I weigh them according to, well, where I should be going, which is going to be informed by these composite nodes down here.

The composite nodes based tell you, where should you be going?

If you're looking for stationary points of your generalized free energy.

Uh, and these nodes, I'm going to say, all right, if this is where I'm going, this is the, the transitions I should take with this and this, this probability.

Yeah.

Okay.

Maybe just to, to clarify and emphasize that what is the policy selection mechanism in direct policy inference?

So that's actually one of the things that I really like about this way of doing it.

You can pick whatever you want.

So we do two here in the paper, one, which is this one, the one that I have on screen here, where you just, you get a posterior out over your policy and you can read off, you know, the posterior reactions at every time step.

And then it's up to you whether you want to sample, what you want to do with it.

But what we also show is that you can add these delta constraints on these edges here that are labeled U.

And if you do, what you get back is a series of map estimates over your policy.

And the trick being that is now something you've write as part of your inference problem.

So as for the, the, the policy selection mechanism, what I'm going to say is we, we should, my opinion, we should be patient about it.

So if we get a distribution over possible policies, that's, that's the way it should be.

And then, you know, you can do what, what you think is right with your distribution.

Yeah, this is a very nice point that the outcome of the EFE iterated across policies is a distribution on policy. And so then you can simply choose the best one. You could choose the worst one. You could cut the deck in the middle. You could do any number of then secondary policy selection policies.

And one that feels natural and makes sense is just to take the first order interpretation of the distribution and then sample directly from that distribution.

You could do that. Or I think a very common thing to do is just say the argmax are either best policy at all time steps, but yeah, either one works. The point being really that, as you said, instead of doing this compute EFE and then construct a distribution of our policies, you just infer the distribution of our policies directly as part of your inference.

Cool.

Yeah. And this actually allows us to do something. We don't really explore too much in the paper,

but which I'm hoping to explore in the future, which is that we get rid of this combinatorial explosion. Because if you look at the kind of things you would have to do, there's a lot of messages going on here, but basically what you should take away is that everything goes back and forth on every edge. And when these two guys collide, they give you the posterior distribution over your policy or over the action at that time step is what it is to say. But because we're reasoning backwards and we don't have to do this for every policy,

we scale linearly in time rather than getting this large explosion.

Like with the caveat that there's some iterations going on and there's some root finding to get these red messages going the right way. There's some extra computation here and there, but this whole procedure pretty much scales linearly in the planning horizon rather than this giant exponential explosion, which to me is really nice because it hints at us being able to do something that's quite a lot bigger than what we have done so far.

Right here on this slide, I'll ask a question from Ryan who wrote, are we guaranteed convergence after a single forwards backwards sweep of the messages? No. So this is an iterative procedure. This function is closer to what you would get with a variational message parsing thing where you have to iterate it a couple of times.

So no, we're not guaranteed convergence after a single forwards backwards pass. Usually that's probably wouldn't be enough. You would have to do like two or three or something.

Ryan Duffy And those iterations, you use some other heuristic, like how much improvement you're getting and is that a monotonic iteration or do you ever...

Ryan Duffy So there's some work still to be done on this.

So I can't claim like 100% mathematical assurity, but this seems to function a lot like a variational message parsing algorithm where things just nicely decrease as you do more iterations. So therefore, more iterations is always better. But it's going to hit like a convergence thing at some point where the gain of running more iterations is going to be marginal. That's how I'm confident this actually works. But because we changed the functional, I'm not sure whether all the convergence proof supplies. I'm not... I can't be 100%. I'm going to put it at like 98.

Ryan Duffy Yeah. You'll never iterate worse. You ratchet and improve. But I guess my question is, could there be drop-offs where you're monotonically improving and then you enter a new regime of another improvement phase?

Ryan Duffy Yeah, I don't see why not. I haven't seen it, but the models we've tested this on have been variations of the T-Maze. They've been quite small.

Ryan Duffy Yeah. So there's still work to be doing actually exploring exactly how this algorithm behaves.

Ryan Duffy Yeah. Not like from theoretical, also from theoretical standpoint, but also just from like the practical, what actually happens if you start running things and you build out models and sort of getting a feel for how these things work. Because doing it in this way is not something that has been done before. At least to my knowledge. Again, I'm going to hedge my bets a little bit here.

Ryan Duffy All right. Are there any more questions?

Ryan Duffy No. Good to carry on.

Ryan Duffy Keep going. Right. Because this is what you actually get out if you run this for a couple of iterations on the T-Maze. So the T-Maze environment is the classic one that we all know and love, but we have

a mouse or agent that starts in this position in the middle here.

And it's looking for a piece of cheese. Piece of cheese is going to be either in this left arm or on this right arm.

However, at the back here is a Q.

And the Q tells the mouse whether the cheese is to the left or to the right.

So the optimal thing to do is to go and visit the queue first and then go check out the correct arm.

And that's sort of the hello world of this epistemic drive that we like with active inference.

Because an active inference agent is going to go and collect information and then go either left or right. Whereas if you just do some greedy, fully goal-directed agent, it's just going to gamble and going either left or right and hoping for the best. So if we do this sort of direct policy inference with this form Lagrange and drive station in point Lagrange and active inference type thing, these are the posteriors we're going to get out. And you see that the agent does actually, in planning only, replicate this nice policy that we want where it first goes and samples the queue.

And afterwards, it's indifferent as to which side to go to, but it knows it's going to go to either of the two sides. And this is something that's done only using planning. So the agent doesn't actually get to execute anything and go and actually sample the queue and update its beliefs.

So in the absence of all that, it can still figure out that, hey, maybe I should go and check the queue first before I go find my snack.

My queue is telling me to find the queue.

Yeah, I mean, pretty much.

So again, when I saw these results, I was very, very happy because this shows that all the sort of heavy duty theoretical work actually paid off and yield something that replicates the standard experiments. That sort of is the baseline that we should be able to do.

All right, what are the next thing we got? Right, yeah. So in part two, we also extend this to do parameter learning. And what that means is we want to learn the A matrix, the likelihood matrix.

And we do that by replicating the standards sort of EFE type algorithm where we inference all of this model and then we compare free energy of the model, which includes these EFE terms for each policy. So we

can sort of get the same type of thing with the policy evaluation and evaluating expected free energy and all the things that we like. And the task is whether we can figure out, like we can solve the task and we can learn this A matrix, what is the actual mapping, the likelihood mapping that we want.

And that's interesting because the first experiment relied mainly on this message, the one flowing out and the output back towards the state. Whereas this one also has a component of this message flowing

towards

a likelihood matrix. Ideally, this should correspond to something like a novelty term in like the classical Schartenbeck formulation, which should drive our agent to go and explore its environment and figure out what's up. Then as we iterate this back and forth a couple of times, we should learn the mapping.

And we see very nicely that free energy decreases the minimum as we learn.

It gets lower and lower and lower. Except for these spikes here. This is when the agent actually gets its observation when it gets its reward, its piece of cheese. That's because there is some noise in the reward probabilities. It's not 100%. So you can see every time it predicts a win but it gets a loss, the free energy spikes.

Which is consistent. If your model predicts something and you get something else, your free energy is going to be off. And if we compare to an agent that does just box standard

Béthi free energy, a variational free energy under the Béthi approximation, we can see that nothing really happens and it loses all the trials because it's greedy. It doesn't go and gather the information like the information at the Q. Furthermore, if we look at the actual sort of matrices that gets learned.

These are the different locations that means the origin, the left arm, the right arm and the Q location.

And this row here indicates whether the Q says go left, Q is go right. And this is whether the reward is on the left or the right. And this is whether you get a reward or whether you get no reward.

Okay. So basically what we can see is that if

the Q is on the right, we also predict that we're going to get the reward on the right.

Get a reward on the right, we predict the reward is on the right.

Get a reward on the right, and so on and so forth.

Get a reward on the left, so on the left.

Basically what this shows is that we are able to accurately learn

the mapping in the environment, which is kind of nice because again, this means that our agent will go and figure things out and actually learn how to navigate an environment that it doesn't know.

Okay. So this is the final slide that I have and I've been talking forever now.

And the main sort of take home message that I like to leave everyone with is

we can put active inference on graphs.

But we can do that and that is awesome.

And once we do, we are left with sort of this Lego block approach to constructing active inference agents. Because everything is local on the graph, I can do my local manipulation to get my EFE, my GFE down here. And then I can build out whatever graph I want in all other directions. I can have hierarchies, I can have temporal depth, I can do smoothing, filtering, I can do control, I can do any combinations of variables that I like.

As long as I can figure out the inference, it all works.

And we can get the classical things that we all know and love as special cases.

And we can solve inference on these using standard tools from belief propagation or variational message parsing,

combined with these new messages that flow out of nodes that up to my GFE.

So this gives us, at least in my mind, a flexible way to construct these large, sprawling freeform models.

Which in the end gets us the best of both worlds as we started with.

So we got the epistemics, we get freeform models, and we have a recipe for doing efficient inference.

And that is everything I got.

Excellent. Well, let's have a little bit more conversation and kind of ask a few questions

as we land the plane on the dot one. And then next time in the dot two, we'll return with maybe a little code, a little bit of drawing, and hopefully some more people on the stream.

Okay. So what does synthetic mean to you?

In this case, it was sort of to underline that this is from an engineering point of view.

You know, I didn't really talk anything about brains or Bayesian mechanics or

individuation and niche construction, all these other really, really interesting things that people are doing with active inference. This is purely like, how do you build a system that does this?

So the synthetic came out of trying to say, well, this is us trying to build engineering tools for active inference.

Very fascinating. Just in how you framed it there, it made me think about the engineer of a bridge. And on one hand, they have colleagues in physics and metallurgy and gravity and the basic physical phenomena.

And then on the other hand, they have colleagues who have bodies, may care about the ergonomics, the cultural components of the architecture. And so it's like engineering actually intermediates between, in this case, the Bayesian mechanics, and on the other hand, biological systems of interest for active inference.

Yeah, you could say that, right? Because this is just a way of doing active inference on graphs.

If you want to come along and be like, okay, I think I can use this to model something about the brain, or I think I use this to model something about interpersonal relationships or whatever. That's amazing.

That is awesome. All the power to you. But what you choose to do with the tools is different to the tools itself.

Like if you have a hammer, you can use it to carve out a beautiful sculpture, you can use it to build a bridge.

But a good hammer is still going to be a good hammer. And all we're doing here is building hammers.

Cool. Okay, another question. You mentioned these sprawling graphs. So how could you connect what is enabled with this work to ecosystems of shared intelligence,

and maybe even like the internet of things like IoT?

So that's one of the things that I would like to see happen at some point. Because if you have

the representation of inference problem as being distributed over a graph, and you can do things

asynchronously when you pass messages, you can run these nodes anywhere. This is just an abstract thing.

So I can have, say, a drone flying around that runs its own generative model and sends messages back to some other thing.

That then sends messages back to some other thing that communicates with my phone and my fridge and my pacemaker.

But as long as all the messages sort of are taken care of and all the little local computations work,

then effectively, it's going to be like just doing inference on a really, really big model.

So one of the things that we wanted to do with the designing ecosystems of intelligence paper is sort of try and push this idea that intelligence is something that does inference and react, probably to its environment.

In nature, it's a distributed thing.

The whole of humanity is way more intelligent than I will ever be. Many, many people are.

And we're trying to, scientists, put stuff into this shared pool of knowledge.

But even the intelligence that I claim is inside my head is really made up of a network of neurons that are interacting and doing things together. And each neuron is made up of a whole bunch of component parts and so

on and so forth. Or, you know, like ants doing interesting things when they swarm around to coordinate.

Most of the types of intelligence that we see in nature is actually distributed already.

And moving to this idea of distributed inference and distributed message parsing on very, very large graphs that do things asynchronously,

is sort of our attempt to saying, well, if intelligence is inherently distributed, we can cast intelligence as this type of inference problem.

We should build our architects, we should build our intelligences in a way that matches this.

We should build them as big distributed systems. And once they are big distributed systems, we should get all the, I mean, then we can use all these tools to figure out what all the local parts do and make sure that you don't have to write out a schematic for the whole system.

Just take care of everything locally around your little bit and all the communications. And if everybody does that, then we're good.

Well, I heartily concur with the distributed intelligence. And when I see this kind of engineering work, it's like the idea that all intelligence is distributed. It's not the end point. It's not a hot take. It's not where the discussion ends. It's like the first rays of light of the discussion beginning.

And so that's why there's so much to do on the formal side, but obviously on the application side.

Okay. Next questions from the chat.

Wotter asks, how straightforward is it to develop algorithms along this perspective? Can I build an agent within a day?

Yes, you can. Because we put this in RxNFIR so far.

It's not in the mainline branch, but the constructions, like the nodes that we used to do our experiments are publicly available.

There's a repo at [github.com slash bioslab slash live](https://github.com/bioslab/live), L-A-I-F, where you can find these things in the interface with RxNFIR, the toolbox that's developed in BiosLab.

So, yeah, the tools are there. They need to be polished a little bit, but they're there.

You can build an agent in a day if you know how to work with RxNFIR.

Otherwise, it might take you two because you spend one work through the demos.

Yeah. I mean, could you maybe go into a little more about what BiosLab is doing with Julia and reactive message passing RxNFIR?

Yeah, of course. So, caveat, there are people who are better at RxNFIR than I am.

Dmitry Bagaiev is sort of the main architect behind it and is a beast of a programmer.

So, the majority of the grunt work has been done by other really, really talented people in BiosLab.

So, RxNFIR is a toolbox for doing message passing based inference, minimizing constraint, battery-free energy, and factor graphs.

And it does so by marrying message passing and reactive programming.

Reactive programming is the idea that instead of you...

Actually, we can do this a little bit here.

Instead of doing something like this, where I pre-specify a schedule, where I write out all the messages that flow on my graph,

I can say, well, any node is just going to listen to all the incoming edges and say, if I can send a message, I will send a message.

If I can send a message, I'll just wait for all the information to be available until I say something.

And if you do that, and you can do that in sort of a reactive way, that basically gives you a way to specify a problem as your inference problem with your constraints and your factor graph and everything.

Hook it up to a sensor in the ideal world.

Hook it up to a sensor.

And then whenever you get observations coming in, that will trigger your inference just when it happens.

And the sort of way that things flow on the graph will be data-driven rather than something that I, as an engineer, specify.

But again, it's something that I like because the less work I have to do, it is for me to get stuff done.

Yeah.

Even as a non-Julia expert, I believe this is a massive step for active inference.

Even with message passing, you're going to be in a situation, if you can't use reactive message passing, where if you have two sensors of different frequency, you either choose to basically have to waste clock cycles calculating updates at a faster rate than the slower sensor, or coarse-grain or throwaway samples from the faster sensor.

So having this kind of just-in-time logistical information supply chain has very, very large performance improvements with some or all of the same formal guarantees.

And it's going to be especially important if there's an ecosystem of shared intelligence where components are active at different times or have asynchronous or that kind of concurrency in effect, which is just basically real systems.

Yeah, exactly.

Because you and I don't have a set schedule and a set way that inference is done.

And we react to what comes in and then we plan and then we do things sort of in real time.

And the beauty of doing it in this way and when everything is local is that even as you go to larger graphs with things happening asynchronously, you inherit a lot of the same convergence guarantees.

So basically, as you run more messages, your inference or your free energy is never going to get worse.

It might, you know, something might happen like we saw these spikes if you get information, data that's not

predicted.

But as you run more inference, it's never going to get any worse.

So you can run this part a little faster and this part a little slower.

And, you know, so it's still going to end up having this nice, nice smooth curve.

Yeah, it's kind of like a net that ratchets everywhere.

Yeah, like it's a large distributed system that reacts to data and reacts to the world as things happen.

And an RxInfer is just being an engineering tool for building that.

If you have that as your vision, you need something that does it.

And that's what RxInfer is attempting to be.

Cool.

So maybe one more question and then we'll close for that one.

Okay.

Upcycle Club writes,

Simulated annealing, including quantum simulated annealing, is considered a meta-heuristic.

Do you use any types of meta-heuristics in synthetic active inference agents?

Not yet.

Not yet.

I'm not familiar with quantum simulated annealing, so I can't really speak to what it does.

But the thing that's different about this approach to active inference as opposed to many other approaches is basically that we can do things locally on the graph and we have these backwards messages.

I'm sure you can combine that with a quantum simulated annealing approach somehow.

And if you find a nice way of doing it, I would love to see it.

But it's not something we've done yet now.

Cool.

Well, this was awesome.

It was very relaxing and even-handed.

Even though it builds on so much.

So it was an epic presentation and I think sets us up really well for .2 when we talk about what happens next now that the .1 is in the books.

Yeah, I had a great time.

And I'm glad you found it relaxing because I think it was a bit more stressful over here.

Maybe receiving the messages is easier than sending them maybe.

Yeah, but I thank you.

This was great.

I thoroughly enjoyed this.

Cool.

So thank you everybody for watching and leave a comment if you have a question.

Or if you're around in time, then maybe get in touch and join 55.2.

So thank you, Magnus.

Farewell.

Thank you.

Bye.

Bye.

Thank you.